

On this page

Monitoring RabbitMQ



For a complete reference of the monitoring recommendations for RabbitMQ, refer to the [official documentation](#). Make sure to refer to the version being used.



The details in this page may change depending on how you have installing your RabbitMQ cluster. For example, if you are using Kubernetes, you should be using the [RabbitMQ Kubernetes Operator](#), and [different recommendations apply](#).

Collecting metrics

RabbitMQ exposes several application-specific metrics that are collected and reported by RabbitMQ nodes. Some of these are cluster-wide, and others are node-specific. It is important to consider this when collecting and storing them.

The recommended monitoring approach from RabbitMQ is using the [Prometheus plugin](#). Feedzai has also used the [management plugin](#) to serve metrics, and also allowing the cluster to be managed via a UI or an API. When activated, the management plugin provides an [HTTP API](#).



There are some performance concerns with the management plugin, which is why the recommendation is to use the Prometheus plugin. If you just wish to monitor cluster-wide metrics (see below), having the plugin enabled on a single node would suffice. However, for redundancy and node-specific metrics, all nodes in the cluster should have it enabled.



Telegraf provides an [input plugin](#) that relies on the RabbitMQ Management Plugin.

Top Metrics to Monitor

In this section you will find which metrics are important to monitor in **RabbitMQ**.



Note that the following list is simply a reference to the most important metrics, and should not be considered as the single source of monitoring the healthiness of the system.

Cluster-Wide Metrics

Cluster-wide metrics provide a high level view of cluster state, including the interaction between cluster nodes. To access these metrics, use the endpoint `GET /api/overview` from the [HTTP API](#)

Metric Name	Description
<code>object_totals.connections</code>	Total number of connections
<code>object_totals.channels</code>	Total number of channels
<code>object_totals.queues</code>	Total number of queues

Metric Name	Description
object_totals.consumers	Total number of consumers
queue_totals.messages_ready	Number of messages ready for delivery
queue_totals.messages_unacknowledged	Number of unacknowledged messages
message_stats.publish_details.rate	Message delivery rate
message_stats.deliver_get_details.rate	Message publish rate



Consider capturing [message metrics individually](#) for the most important queues with the `GET /api/queues/{vhost}/{qname}` endpoint.

Node Metrics🔗

Most of the metrics represent point-in-time absolute values of individual nodes. To access these metrics, use the endpoint `GET /api/nodes/{node}` from the [HTTP API](#) (or `GET /api/nodes` if your collector supports an array).

Metric Name	Description
proc_total	Erlang process limit
proc_used	Erlang processes used
mem_limit	Memory usage high watermark
mem_used	Total amount of memory used
queue_totals.messages_ready	Number of messages ready for delivery
queue_totals.messages_unacknowledged	Number of unacknowledged messages